# 一、先行进位的构建思路

## 1.1 考虑打破进位依赖

1. ***约定符号***

**构建先行进位电路**

$S_i = X_i \oplus Y_i \oplus C_i$

$C_{i+1} = X_iY_i + (X_i \oplus Y_i)C_i$

令 $G_i = X_iY_i$，$P_i = X_i \oplus Y_i$

$G_i$ 称为**进位生成函数**，$P_i$ 称为**进位传递函数**

$S_i = P_i \oplus C_i$

$C_{i+1} = G_i + P_iC_i$

当 $X_i = 1$，$Y_i = 1$时 ➡ $\begin{cases} G_i = 1 \cdot 1 = 1 \\ P_i = 1 \oplus 1 = 0 \end{cases}$ ➡ $C_{i+1} = 1 + 0 \cdot C_i = 1$

当 $X_i \neq Y_i$时 ➡ $\begin{cases} G_i = 0 \\ P_i = 1 \end{cases}$ ➡ $C_{i+1} = 0 + 1 \cdot C_i = C_i$

2. ***写出递归表达式***

**构建先行进位电路**

$S_i = P_i \oplus C_i$

$C_{i+1} = G_i + P_iC_i$

$G_i$ 称为**进位生成函数**，$P_i$ 称为**进位传递函数**

令 $G_i = X_iY_i$，$P_i = X_i \oplus Y_i$

$S_i = X_i \oplus Y_i \oplus C_i$

$C_{i+1} = X_iY_i + (X_i \oplus Y_i)C_i$

$C_1 = G_0 + P_0C_0$

$C_2 = G_1 + P_1C_1 = G_1 + P_1(G_0 + P_0C_0) = G_1 + P_1G_0 + P_1P_0C_0$

$C_3 = G_2 + P_2C_2 = G_2 + P_2(G_1 + P_1G_0 + P_1P_0C_0) = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$

$C_4 = G_3 + P_3C_3 = G_3 + P_3(G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0) = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$

$\vdots$

$C_n = G_{n-1} + P_{n-1}G_{n-2} + P_{n-1}P_{n-2}G_{n-3} + \cdots + P_{n-1}P_{n-2}\cdots P_1P_0C_0$

**向高位的进位输出$C_n$由 $G_i$、 $P_i$以及$C_0$经过逻辑运算得到， $C_n$不再依赖于$C_{n-1}$。**

3. ***画出组合电路图***

$C_{i+1} = G_i + P_i C_i$　　令 $G_i = X_i Y_i$，$P_i = X_i \oplus Y_i$

$C_1 = G_0 + P_0 C_0$

$C_2 = G_1 + P_1 C_1 = G_1 + P_1(G_0 + P_0 C_0) = G_1 + P_1 G_0 + P_1 P_0 C_0$

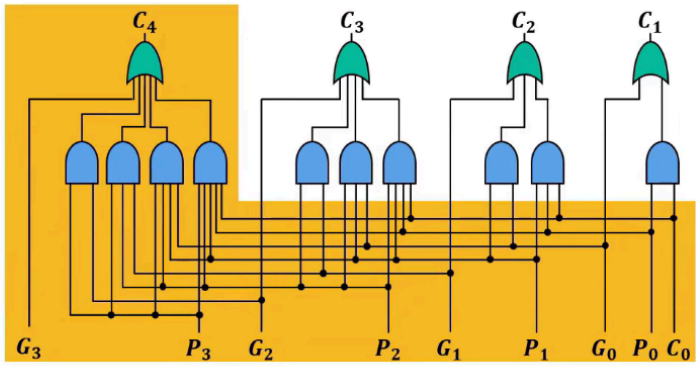
$C_3 = G_2 + P_2 C_2 = G_2 + P_2(G_1 + P_1 G_0 + P_1 P_0 C_0) = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$

$C_4 = G_3 + P_3 C_3 = G_3 + P_3(G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0) = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$

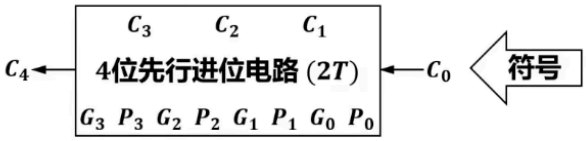
画出相应的组合逻辑电路

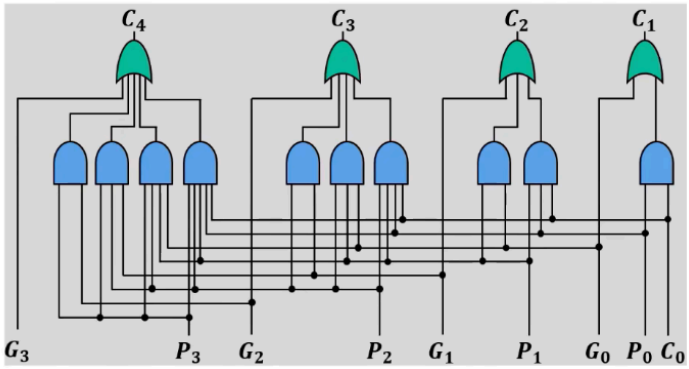
$\vdots$

$C_n = G_{n-1} + P_{n-1} G_{n-2} + P_{n-1} P_{n-2} G_{n-3} + \cdots + P_{n-1} P_{n-2} \cdots P_1 P_0 C_0$




## 基于4位先行进位电路构建4位快速加法器

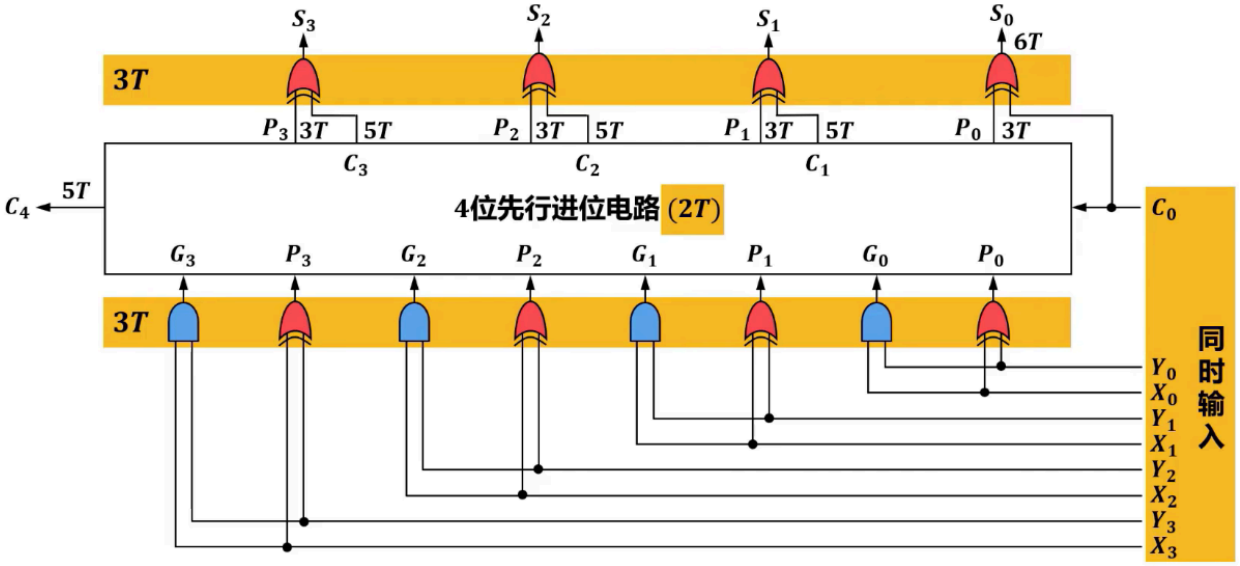



4. *四位快速加法器电路图*

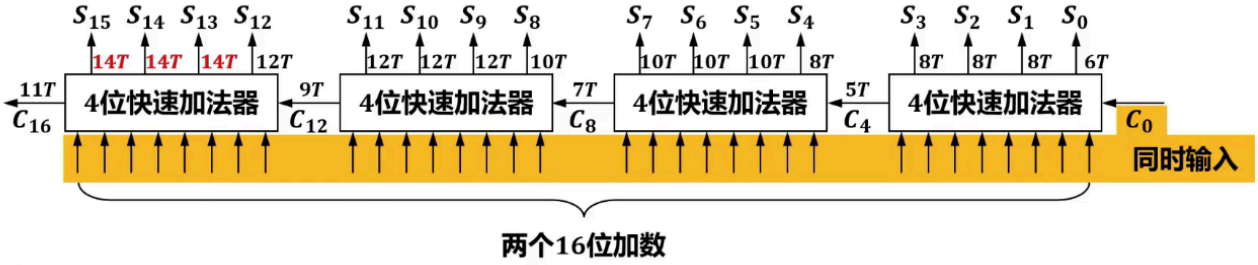
## 基于4位先行进位电路构建4位快速加法器

$S_i = P_i \oplus C_i$　　$C_{i+1} = G_i + P_i C_i$　　令 $G_i = X_i Y_i$，$P_i = X_i \oplus Y_i$



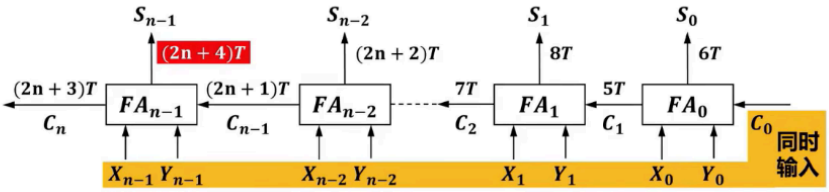

5. *快速加法器与串行加法器性能对比*

### 使用4位快速加法器构建更大位宽的加法器电路




**上述加法器电路的时间关键延迟为：** $14T$

$S_{n-1}$**的时间延迟为:** $(2\times 16+4)T=36T$




## 二、基于基础先行进位加法器的优化

相比串行进位器，上图的优点是实现了**组内并行**，但**组间依旧是串行**的，所以我们考虑是否组间也可以实现并行：***可级联的先行进位加法器构建***

### 2.1 符号约定

#### 可级联的4位先行进位电路

$C_{i+1}=G_i+P_iC_i$　　令 $G_i=X_iY_i$，$P_i=X_i\oplus Y_i$

$C_1=G_0+P_0C_0$

$C_2=G_1+P_1C_1=G_1+P_1(G_0+P_0C_0)=G_1+P_1G_0+P_1P_0C_0$

$C_3=G_2+P_2C_2=G_2+P_2(G_1+P_1G_0+P_1P_0C_0)=G_2+P_2G_1+P_2P_1G_0+P_2P_1P_0C_0$

$C_4=G_3+P_3C_3=G_3+P_3(G_2+P_2G_1+P_2P_1G_0+P_2P_1P_0C_0)=G_3+P_3G_2+P_3P_2G_1+P_3P_2P_1G_0+P_3P_2P_1P_0C_0$

**画出相应的组合逻辑电路**

$C_n=G_{n-1}+P_{n-1}G_{n-2}+P_{n-1}P_{n-2}G_{n-3}+\cdots+P_{n-1}P_{n-2}\cdots P_1P_0C_0$

$C_4=G_3+P_3G_2+P_3P_2G_1+P_3P_2P_1G_0+P_3P_2P_1P_0C_0$

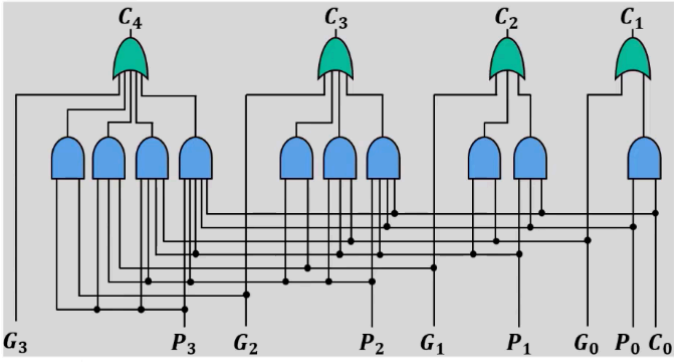
记为 $G^*$　　记为 $P^*$

形式相同

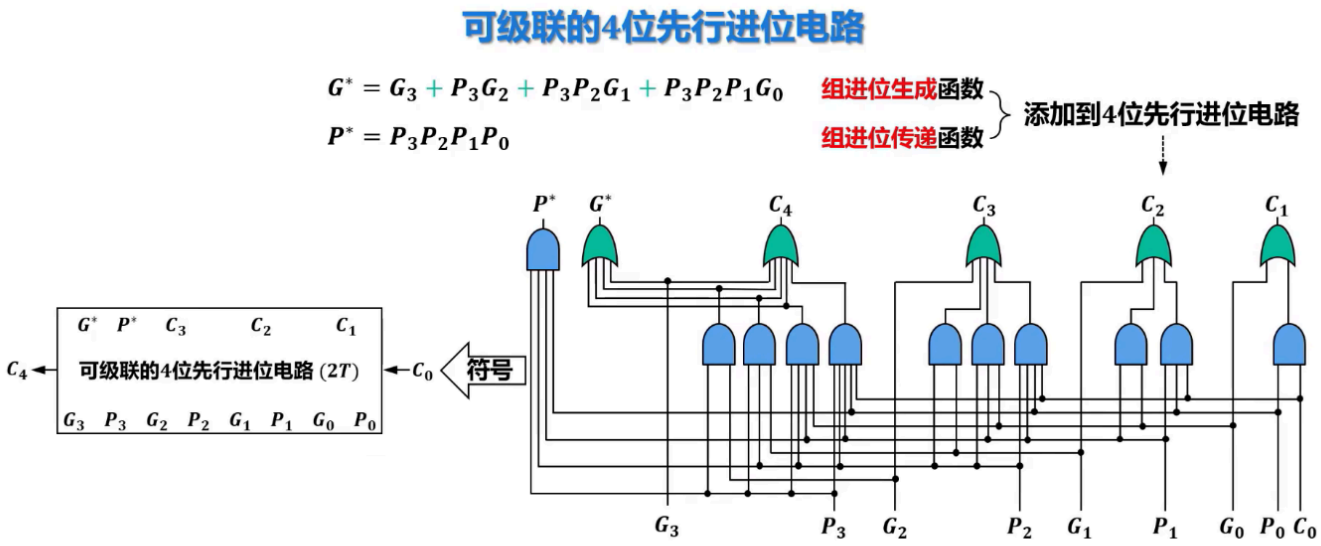
$C_4=G^*+P^*C_0$

$G^*=G_3+P_3G_2+P_3P_2G_1+P_3P_2P_1G_0$　组进位生成函数

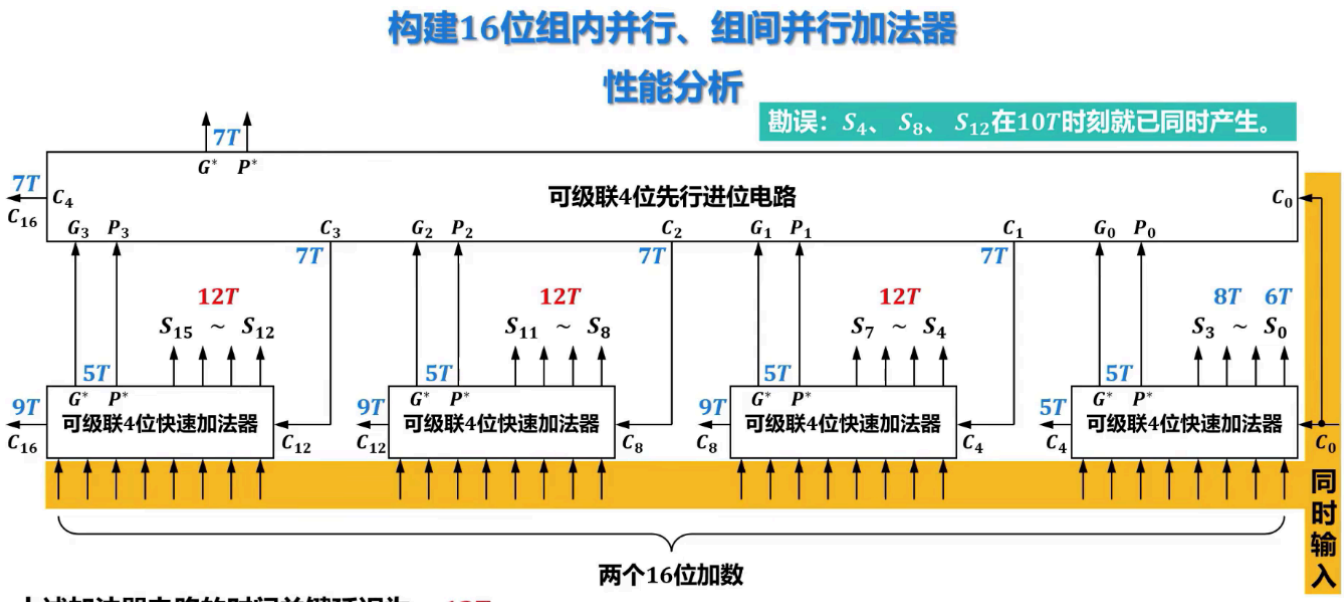
$P^*=P_3P_2P_1P_0$　组进位传递函数



## 2.2 添加了==组进位生成函数==和==组进位传递函数==的四位先行进位加法器




## 2.3 十六位==组间与组内皆并行==的加法器




**上述加法器电路的时间关键延迟为： 12T**

==***结构详解***==

### 1. 先行进位的核心思想：进位其实可以提前"猜"到

**两个关键信号：G 和 P**

对于单独的一位（比如第 i 位），看它的两个输入 Ai 和 Bi：

| Ai | Bi | 情况 | 进位结果 | |
|---|---|---|---|---|
| 1 | 1 | **不管低位有没有进位，这一位一定会产生进位** | 必然进位 | |
| 0 | 0 | **不管低位有没有进位，这一位一定不产生进位** | 必然不进位 | |
| 1 | 0 或 0 | 1 | **如果低位有进位，我就传过去；没有就不传** | 取决于低位 |

于是数学家定义了两个信号：

- **G（Generate，生成）**：这一位自己就能"生"出一个进位。条件：Ai 和 Bi 都是 1
- **P（Propagate，传播）**：这一位不会自己产生进位，但会把低位的进位"传"上去。条件：Ai 和 Bi 中至少有一个是 1

> **通俗理解**：G 就像"这层楼自己着火了"，P 就像"这层楼有消防通道，能把楼下的火引上来"

#### 进位的真正公式

第 i 位的进位输出可以写成：

$$C_{i+1} = G_i + P_i \cdot C_i$$

意思是：**要么这一位自己生火（G），要么低位有火且我这层能传火（P·C）**

这个公式的美妙之处在于：**Ci+1 的值完全由输入决定，不需要真的等前面的位"算完"！**

---

### 2. 为什么要"分组"？

如果 16 位全部用上面的方法展开，逻辑会复杂到爆炸。所以工程师想了个办法：

**把 16 位切成 4 组，每组 4 位。组内用先行进位快速算，组间再用另一套先行进位快速算**

#### 每组输出"打包报告"：G* 和 P*

每个 4 位模块内部，根据那 8 个输入（4 位被加数 + 4 位加数），算出两个"总结报告"：

- **G***（组生成）：这 4 位内部**自己就会产生一个进位**，而且这个进位能传到这 4 位的最高位
- **P***（组传播）：如果低位给这 4 位一个进位，这 4 位**能把它传到最高位**

> **类比**：G* 就像小组长报告"我们组内部出问题了"，P* 就像"我们组能把上级的问题传下去"
> **关键：G* 和 P* 只取决于这 4 位的输入，和外部进位无关！** 所以它们可以**立刻**算出来

---

### 3. 看图说话：两级结构详解

现在回到你的图，从上到下、从右到左看：

#### 第一级：下方四个"可级联4位快速加法器"

这是真正干活的地方：

| 模块（从右到左） | 负责位数 | 输入进位 | 输出进位 | 输出和 |
| --- | --- | --- | --- | --- |
| 第 0 组 | $S_3$ ~ $S_0$（最右边4位） | $C_0$ | $C_4$ | $S_3$~$S_0$ |
| 第 1 组 | $S_7$ ~ $S_4$ | $C_4$ | $C_8$ | $S_7$~$S_4$ |
| 第 2 组 | $S_{11}$ ~ $S_8$ | $C_8$ | $C_{12}$ | $S_{11}$~$S_8$ |
| 第 3 组 | $S_{15}$ ~ $S_{12}$（最左边4位） | $C_{12}$ | $C_{16}$ | $S_{15}$~$S_{12}$ |

每个模块上方都引出两根线 **G*** 和 **P***，送到顶部的横条

##### 第二级：上方"可级联4位先行进位电路"

这是一个**纯组合逻辑电路**（只有门电路，没有记忆），它做一件事：

**接收**：
- 初始进位 $C_0$（最右边进来的）
- 4 个模块的 G*、P*（共 8 根线）

**输出**：
- $C_1$、$C_2$、$C_3$、$C_4$（注意：这是**上方电路自己的编号**）

##### 这里就是你之前困惑的地方！

**上方电路的 $C_1$、$C_2$、$C_3$、$C_4$ 分别对应下方模块之间的进位：**

| 上方输出 | 实际送到哪里 | 就是 |
| --- | --- | --- |
| $C_1$ | 第 1 个模块的输入 | **$C_4$** |
| $C_2$ | 第 2 个模块的输入 | **$C_8$** |
| $C_3$ | 第 3 个模块的输入 | **$C_{12}$** |
| $C_4$ | 最左边溢出 | **$C_{16}$** |

> 为什么编号不一样？下方按"位数"叫（第4位、第8位……），上方按"第几个组间进位"叫（第1个、第2个……）。**本质上是一根线**

---

### 4. 展开讲：进位是怎么"飞"过去的？

这是最难理解的部分，我用一个**快递分拣中心**的类比来讲

##### 传统串行进位 = 快递站接力

16 个快递站排成一排。第 0 站收到包裹处理完，传给第 1 站；第 1 站处理完传给第 2 站……第 15 站必须等前面 15 个站全部忙完

##### 先行进位 = 所有站同时填表，调度中心统一派单

#### 第一步：各站同时填"自评表"（第一级并行）

4 个模块（4 个快递小组）同时开始工作：
- 各自看自己的 8 个输入（4 位加数 + 4 位被加数）
- 同时算出：

1. 自己那 4 位的和（S）
2. 自己的 G*（我们组会不会自己出进位）
3. 自己的 P*（我们组能不能传进位）

**注意：这时候它们还不知道外部进位是多少！** 但 G* 和 P* 不需要知道外部进位就能算。

**第二步：调度中心看所有自评表，同时算出所有组间进位（第二级并行）**

上方先行进位电路拿到 4 份"自评表"（$G_0$*~$G_3$*, $P_0$*~$P_3$*）和初始进位 $C_0$，用逻辑门**同时**算出：

$$C_4 = G_0^* + P_0^* \cdot C_0$$

$$C_8 = G_1^* + P_1^* \cdot G_0^* + P_1^* \cdot P_0^* \cdot C_0$$

$$C_{12} = G_2^* + P_2^* \cdot G_1^* + P_2^* \cdot P_1^* \cdot G_0^* + P_2^* \cdot P_1^* \cdot P_0^* \cdot C_0$$

$$C_{16} = \text{（类似，更长，但逻辑一样）}$$

**为什么 $C_8$ 的公式里有 $G_0$* 和 $P_0$*，但不需要等第 0 组算完?**

因为**第 0 组和第 1 组是同时填自评表的**！$G_0$* 和 $G_1$* 几乎是同时产生的。上方电路等所有 G*、P* 都到齐后（只是一瞬间），门电路的电流"唰"地流过去，所有进位**同时**就出来了。

**第三步：把进位发回各模块，完成最终计算**

上方算出的 $C_4$、$C_8$、$C_{12}$ 分别送回第 1、2、3 个模块。这些模块拿到进位后，稍微调整一下内部的和（因为组内也是先行进位结构，早就准备好了，只等这个外部进位"选"一下最终结果），就输出正确的 S。

---

### 5. 时间线：信号流的全过程

假设一个逻辑门延迟是 1 个单位时间：

| 时间点 | 发生了什么 |
|---|---|
| **T=0** | 16 位加数和被加数同时到达 |
| **T=1~2** | 4 个模块**同时**计算：各自内部的 G、P、G*、P*、以及"假设进位为0和1时的两种可能结果" |
| **T=3** | 上方先行进位电路拿到所有 G*、P*，经过与门/或门，**同时**算出 $C_4$、$C_8$、$C_{12}$、$C_{16}$ |
| **T=4** | 各模块收到组间进位，选出正确的和，输出 $S_0$~$S_{15}$ |

**总共约 4 个门延迟！**

而如果是传统串行进位：16 位需要约 **16~32 个门延迟。**

---

### 6. 为什么叫"可级联"?

注意图中最左边和最右边还有进位线：
- 最右边输入 $C_0$，最左边输出 $C_{16}$
- 最左边还输出 G* 和 P*

这意味着：**你可以把多个这样的 16 位加法器再拼起来！**

比如两个这样的 16 位单元可以拼成 32 位加法器：
- 右边 16 位的 $C_{16}$ 作为进位传给左边 16 位的 $C_0$
- 或者更高级地，把两组的 G*、P* 再送到更上层的先行进位电路

这就是**第三级先行进位。**你的图是两级（组内一级 + 组间一级），还可以有三级、四级.....